PCET Education Trust

Pimpri Chinchwad College OF Engineering, Pune

Database Management Systems Lab

ASSIGNMENT NO. 07

Name	PRN
Atharva Lokhande	124B1F009

Batch A1

Guided By: Mrs. Alpana Borse Ma'am

Aim:

Design and Develop Queries using CRUD operations. (Use CRUD operations, SAVE method, logical operators, joins etc.).

Objectives:

- **To implement NoSQL CRUD operations** by designing and developing queries for document insertion, retrieval, modification, and deletion within a non-relational database environment.
- **To utilize MongoDB logical and comparison operators** such as $and, $or, $gt, and $in to perform advanced data filtering on complex datasets.
- **To apply data modification techniques** using atomic operators like $set, $inc, and $rename, along with replaceOne and upsert logic to maintain dynamic records.
- **To execute relational-style logic in NoSQL** by using variables and distinct value arrays to perform manual joins between collections for both the Airline and Civic Voice systems.

Case Study: Airline Management System

SQL Queries:

Database Initialization

Dropping existing collections ensures a clean state before defining the schema for airlines, flights, passengers, and bookings.

```
test> use airlines switched
to db airlines airlines>
db.airlines.drop() true
airlines> db.flights.drop()
true
airlines>
db.passengers.drop() true
airlines> db.bookings.drop()
true
airlines> db.createCollection("airlines")
{ ok: 1 } airlines>
db.createCollection("flights")
{ ok: 1 } airlines>
db.createCollection("passengers")
{ ok: 1 } airlines>
db.createCollection("bookings")
{ ok: 1 }
```

Data Insertion

Populating the collections using *insertMany and insertOne* to establish the initial dataset for the airline system.

```
airlines> db.airlines.insertMany([
|    { _id: 1, name: "Air India", country: "India" },
|    { _id: 2, name: "Indigo", country: "India" },
|    { _id: 3, name: "SpiceJet", country: "India" },
|    { _id: 4, name: "Emirates", country: "UAE" }
| ])
{ acknowledged: true, insertedIds: { '0': 1, '1': 2, '2': 3, '3': 4 } } airlines>
db.flights.insertMany([
|    { _id: 101, flight_name: "AI101", source: "Mumbai", destination: "Delhi",
```

```
price: 5500.00, airline_id: 1 },
|    { _id: 102, flight_name: "IN202", source: "Pune", destination: "Bangalore",
price: 4500.00, airline_id: 2 },
|    { _id: 103, flight_name: "SJ303", source: "Delhi", destination: "Goa", price:
6000.00, airline_id: 3 }
| ])
{ acknowledged: true, insertedIds: { '0': 101, '1': 102, '2': 103 } }
airlines> db.passengers.insertMany([
|    { _id: 1, pname: "Rahul Sharma", gender: "M", age: 28 },
|    { _id: 2, pname: "Neha Verma", gender: "F", age: 25 },
|    { _id: 3, pname: "Amit Patil", gender: "M", age: 32 }
| ])
{ acknowledged: true, insertedIds: { '0': 1, '1': 2, '2': 3 } }
airlines> db.flights.insertOne({
|    _id: 104, flight_name: "EM404", source: "Dubai", destination: "Mumbai", price:
15000.00, airline_id: 4
| })
{ acknowledged: true, insertedId: 104 }
```

Upsert Operations

Using *replaceOne* with the *upsert: true* flag allows you to either update an existing document or create a new one if no match is found.

```
airlines> db.airlines.replaceOne(
|    { _id: 5 },
|    { _id: 5, name: "Akasa Air", country: "India" },
|    { upsert: true }
| )
{
  acknowledged: true,
insertedId: 5,
matchedCount: 0,
modifiedCount: 0,
upsertedCount: 1
}
airlines> db.bookings.replaceOne(
|    { _id: 3 },
|    { _id: 3, passenger_id: 4, flight_id: 104, seat_no: "8D", status: "Confirmed"
},
|    { upsert: true }
| )
{
  acknowledged: true,
```

```
  insertedId: null,  matchedCount: 1,  modifiedCount: 1,
upsertedCount: 0
}
```

Advanced Querying

Utilizing comparison operators like *$gt* (greater than) and logical operators like *$or and $and* to filter specific datasets.

```
airlines> db.flights.find({ price: { $gt: 5000 } })
[
  { _id: 101, flight_name: 'AI101', source: 'Mumbai', destination: 'Delhi', price:
5500, airline_id: 1 },
  { _id: 103, flight_name: 'SJ303', source: 'Delhi', destination: 'Goa', price:
6000, airline_id: 3 },
  { _id: 104, flight_name: 'EM404', source: 'Dubai', destination: 'Mumbai', price:
15000, airline_id: 4 }
]
airlines> db.flights.find({
|    $or: [ { source: "Mumbai" }, { destination: "Mumbai" } ]
| })[
  { _id: 101, flight_name: 'AI101', source: 'Mumbai', destination: 'Delhi', price:
5500, airline_id: 1 },
  { _id: 104, flight_name: 'EM404', source: 'Dubai', destination: 'Mumbai', price:
15000, airline_id: 4 }
]
```

Field and Value Updates

Performing atomic updates using *$set* to change values, *$inc* for increments, and *$rename* to modify field keys.

```
airlines> db.passengers.updateMany(
|    { age: { $lt: 30 } },
|    { $inc: { age: 1 } }
| )
{ acknowledged: true, insertedId: null, matchedCount: 3, modifiedCount: 3,
upsertedCount: 0 }

airlines> db.passengers.updateOne(
|    { _id: 3 },
|    { $rename: { "pname": "full_name" } }
```

```
| )
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1,
upsertedCount: 0 }
```

Deletion and Document Retrieval

Removing data based on specific criteria and using variables to perform manual joins between collections.

```
airlines> db.bookings.deleteOne({ _id: 2 })
{ acknowledged: true, deletedCount: 1 }

airlines> var myAirline = db.airlines.findOne({ name: "Indigo" })
airlines> db.flights.find({ airline_id: myAirline._id })
[
  { _id: 102, flight_name: 'IN202', source: 'Pune', price: 4500, airline_id: 2 }
]

airlines> var confBookings = db.bookings.distinct("passenger_id", { status:
"Confirmed" })ledged: true, deletedCount: 1 }

airlines> var myAirline = db.airlines.findOne({ name: "Indigo" })
airlines> db.flights.find({ airline_id: myAirline._id })
[
  { _id: 102, flight_name: 'IN202', source: 'Pune', price: 4500, airline_id: 2 }
]

airlines> var confBookings = db.bookings.distinct("passenger_id", { status:
"Confirmed" }) airlines> db.passengers.find({ _id: { $in:
confBookings } })
[
  { _id: 1, pname: 'Rahul Sharma', gender: 'M', age: 29 },
  { _id: 4, pname: 'Sneha Kulkarni', gender: 'F', age: 28 }
]
```

Case Study: Civic Voice Interface System SQL Queries:

Collection Initialization

The process begins by clearing existing data and explicitly creating collections to store users, profiles, complaints, and documents.

```
test> use civic_voice
switched to db civic_voice
civic_voice> db.user.drop()
true
civic_voice> db.citizenprofile.drop()
true
civic_voice> db.complaint.drop()
true
civic_voice> db.userdocument.drop()
true
civic_voice> db.createCollection("user")
{ ok: 1 }
civic_voice> db.createCollection("citizenprofile")
{ ok: 1 } civic_voice>
db.createCollection("complaint")
{ ok: 1 } civic_voice>
db.createCollection("userdocument")
{ ok: 1 }
```

Data Population

Using *insertMany and insertOne* to build the initial dataset, representing different roles and associated citizen metadata.

```
civic_voice> db.user.insertMany([
|    { _id: 1, name: "Rahul Sharma", email: "rahul@email.com", role: "Citizen" },
|    { _id: 2, name: "Neha Verma", email: "neha@email.com", role: "Citizen" },
|    { _id: 3, name: "Amit Patil", email: "amit@email.com", role: "Citizen" }
| ])
{ acknowledged: true, insertedIds: { '0': 1, '1': 2, '2': 3 } }

civic_voice> db.citizenprofile.insertMany([
```

```
|    { _id: 101, user_id: 1, state: "Maharashtra", district: "Pune", age: 28 },
|    { _id: 102, user_id: 2, state: "Delhi", district: "New Delhi", age: 25 },
|    { _id: 103, user_id: 3, state: "Maharashtra", district: "Mumbai", age: 32 }
| ])
{ acknowledged: true, insertedIds: { '0': 101, '1': 102, '2': 103 } }

civic_voice> db.complaint.insertMany([
|    { _id: 1, user_id: 1, title: "Pothole on Road", status: "Pending" },
|    { _id: 2, user_id: 2, title: "Broken Streetlight", status: "Pending" }
| ])
{ acknowledged: true, insertedIds: { '0': 1, '1': 2 } }

civic_voice> db.user.insertOne({
|    _id: 4, name: "Priya Singh", email: "priya@email.com", role: "Guest"
| })
{ acknowledged: true, insertedId: 4 }
```

Upsert and Document Replacement

Using *replaceOne with upsert: true* to handle cases where a record might not yet exist, and modifying existing documents entirely.

```
civic_voice> db.citizenprofile.replaceOne(
|    { _id: 104 },
|    { _id: 104, user_id: 4, state: "Karnataka", district: "Bangalore", age: 29 },
|    { upsert: true }
| )
{
  acknowledged: true,
insertedId: 104,
matchedCount: 0,
modifiedCount: 0,
upsertedCount: 1
}
```

Advanced Querying and Logical Operators

Utilizing MongoDB comparison operators *($gt, $lte, $in)* and logical operators *($and, $or, $nor)* to filter the civic data.

```
civic_voice> db.complaint.find({
|    $and: [ { status: "Pending" }, { user_id: 1 } ]
| })
[ { _id: 1, user_id: 1, title: 'Pothole on Road', status: 'Pending' } ]

civic_voice> db.userdocument.find({
|    $nor: [ { is_verified: false }, { doc_name: "PAN Card" } ]
| })
[ { _id: 1001, user_id: 1, doc_name: 'Aadhaar Card', is_verified: true } ]

civic_voice> db.citizenprofile.find({
|    age: { $in: [ 25, 28, 32 ] }
| })
[
  { _id: 101, user_id: 1, state: 'Maharashtra', district: 'Pune', age: 28 },
  { _id: 102, user_id: 2, state: 'Delhi', district: 'New Delhi', age: 25 },
  { _id: 103, user_id: 3, state: 'Maharashtra', district: 'Mumbai', age: 32 }
]
```

Updates and Field Manipulation

Applying structural changes using *$set, $inc, $unset, and $rename* to update roles, ages, and field names.

```
civic_voice> db.user.updateOne(
|    { _id: 1 },
|    { $set: { role: "Moderator" } }
| )
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1,
upsertedCount: 0 }

civic_voice> db.citizenprofile.updateOne(
|    { _id: 103 },
|    { $rename: { "district": "city" } }
| )
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1,
upsertedCount: 0 }
```

Variable-Based Joins and Deletion

Executing relational-style queries by storing user/profile details in variables and cleaning up data with *deleteOne and deleteMany*.

```
civic_voice> var myUser = db.user.findOne({ name: "Rahul Sharma" })
civic_voice> db.citizenprofile.find({ user_id: myUser._id })
[ { _id: 101, user_id: 1, state: 'Maharashtra', district: 'Pune', age: 29 } ]

civic_voice> var activeUsers = db.user.distinct("_id", { role: "Citizen" })
civic_voice> db.userdocument.find({ user_id: { $in: activeUsers } })
[ { _id: 1002, user_id: 2, doc_name: 'PAN Card', is_verified: false } ]

civic_voice> db.user.deleteOne({ _id: 4 })
{ acknowledged: true, deletedCount: 1 }

civic_voice> db.userdocument.deleteMany({ is_verified: false })
{ acknowledged: true, deletedCount: 1 }
```

Conclusion:

Through this assignment, I successfully implemented **CRUD (Create, Read, Update, Delete)** operations using **MongoDB shell** commands. Unlike traditional SQL, MongoDB's document-oriented approach allowed for a flexible schema, which I managed using advanced features like **upserts** and **atomic field updates**.

By utilizing logical operators and manual joins via variables (such as matching airline_id across collections), I demonstrated how to maintain relational logic within a NoSQL structure. The practical application to the **Airline Management System** and **Civic Voice Interface** confirms that MongoDB is a powerful tool for handling diverse, scalable data models while ensuring efficient data manipulation and retrieval.